

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

+ Criação de Dicionários

Métodos de Criação

```
# 1. Dicionário vazio
dicio_vazio = {}
dicio_vazio2 = dict()

# 2. Dicionário com elementos
pessoa = {
    "nome": "Carlos",
    "idade": 35,
    "profissao": "Médico"
}

# 3. Usando dict() com argumentos nomeados
cores = dict(vermelho="#FF0000", azul="#0000FF", verde="#00FF00")

# 4. A partir de lista de tuplas
pares = [("a", 1), ("b", 2), ("c", 3)]
dicio_pares = dict(pares)

# 5. A partir de duas listas (zip)
chaves = ["nome", "idade", "cidade"]
valores = ["João", 25, "Salvador"]
dicio_zip = dict(zip(chaves, valores))
print(dicio_zip) # {'nome': 'João', 'idade': 25, 'cidade': 'Salvador'}
```

Dictionary Comprehension

```
# Criando dicionário com comprehension
# Sintaxe: {chave: valor for item in iterável}

# Quadrados dos números
quadrados = {x: x**2 for x in range(1, 6)}
print(quadrados) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Com condição
pares_quadrados = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(pares_quadrados) # {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

```
# A partir de lista
nomes = ["Ana", "Bruno", "Carla"]
iniciais = {nome: nome[0] for nome in nomes}
print(iniciais) # {'Ana': 'A', 'Bruno': 'B', 'Carla': 'C'}

# Transformando lista em dicionário
notas = [8.5, 7.0, 9.2, 6.8]
notas_dict = {f"prova_{i+1}": nota for i, nota in enumerate(notas)}
print(notas_dict) # {'prova_1': 8.5, 'prova_2': 7.0, 'prova_3': 9.2, 'prova_4': 6.8}
```

Criação Dinâmica

```
# Criando dicionário dinamicamente
contatos = {}

# Adicionando contatos um por um
contatos["João"] = {"telefone": "99999-1111", "email": "joao@email.com"}
contatos["Maria"] = {"telefone": "99999-2222", "email": "maria@email.com"}
contatos["Pedro"] = {"telefone": "99999-3333", "email": "pedro@email.com"}

print(contatos)

# Usando update() para adicionar múltiplos itens
novos_contatos = {
    "Ana": {"telefone": "99999-4444", "email": "ana@email.com"},
    "Bruno": {"telefone": "99999-5555", "email": "bruno@email.com"}
}
contatos.update(novos_contatos)
print(contatos)
```

Dicionários com Dados Estruturados

```
# Criando dicionário com dados estruturados
produtos = {
    "notebook": {
        "preco": 2500.00,
        "estoque": 10,
        "categoria": "Eletrônicos",
        "especificacoes": {
            "processador": "Intel i7",
            "memoria": "16GB",
            "armazenamento": "512GB SSD"
        }
    },
    "mouse": {
        "preco": 50.00,
        "estoque": 25,
        "categoria": "Acessórios",
        "especificacoes": {
            "tipo": "Óptico",
            "conexao": "USB",

```

```

        "dpi": 1600
    }
}

# Acessando dados aninhados
produto = "notebook"
print(f"{produto.title()}:")
print(f"    Preço: R$ {produtos[produto]['preco']:.2f}")
print(f"    Processador: {produtos[produto]['especificacoes']['processador']}")

```

- **Legibilidade:** Use estruturas claras e consistentes



💡 Exemplo: Sistema de Configuração

```

# Sistema de configuração de aplicação
config = {
    "database": {
        "host": "localhost",
        "port": 5432,
        "name": "meuapp",
        "user": "admin",
        "password": "senha123"
    },
    "server": {
        "host": "0.0.0.0",
        "port": 8080,
        "debug": True,
        "ssl": False
    },
    "logging": {
        "level": "INFO",
        "file": "app.log",
        "max_size": "10MB"
    }
}

# Acessando configurações
db_config = config["database"]
print(f"Banco: {db_config['host']}:{db_config['port']}")
print(f"Debug: {config['server']['debug']}")

```

⚠ Cuidados na Criação

- **Chaves duplicadas:** A última chave sobrescreve as anteriores
- **Chaves hashable:** Use apenas tipos imutáveis como chaves
- **Performance:** Dictionary comprehension é mais eficiente